

一、 算术编码的编码与译码：

1.任务说明

要求：输入字符集为{a,b},且 $p(a)=1/4, p(b)=3/4$.对长度 $L \leq 60$ （至少 30）的序列进行算术编码,并进行反向译码

输入文件:in1.txt，含至少两组输入，每组为满足要求的串

输出文件:out1.txt，对每组输入的编码和译码结果

2.问题分析、实现原理、流程图

2.1 问题分析

算术编码是图像压缩的主要算法之一，是一种无损数据压缩方法，也是一种熵编码的方法。和其它熵编码方法不同的地方在于，其他的熵编码方法通常是把输入的消息分割为符号，然后对每个符号进行编码，而算术编码是直接把整个输入的消息编码为一个数，一个满足 $(0.0 \leq n < 1.0)$ 的小数 n 。算术编码是用符号的概率和它的编码间隔两个基本参数来描述的。算术编码可以是静态的或是自适应的。在静态算术编码中，信源符号的概率是固定的。在自适应算术编码中，信源符号的概率根据编码时符号出现的频繁程度动态地进行修改。

算术编码的本质思想，是对于高频的字符进行短编码，实现方法是会根据符号概率对 $[0,1)$ 进行区间划分，然后根据读入原始数据重复进行选取新目标区间和根据符号概率占比再划分区间的过程，直到最后一个字符。当然，为了使最终二进制编码更短，就需要使得最终目标区间的范围更大。为了使最终目标区间的范围更大，就需要赋予高频字符更大的区间，低频字符更小的区间。

而其译码的方法显而易见，就是编码过程的逆推。从编码得到的小数开始，不断地寻找小数落在了哪个概率区间，就能将原来的字符一个个地找出来。

用一个例子来展示这个过程：

对于一段原始数据字符：AABABCABAB

$P(A) = 0.5 \quad P(B) = 0.4 \quad P(C) = 0.1$

编码：

则算术编码过程会对区间 $[0,1)$ 进行划分：A:[0, 0.5), B:[0.5, 0.9), C:[0.9, 1)

原始数据字符串的第 1 个字符为：A，那么我们选中了 A 的区间 $[0, 0.5)$ 作为新的目标区间。我们对新目标区间，再按照 ABC 的概率占比进行划分：

A:[0, 0.25), B:[0.25, 0.45), C:[0.45, 0.5)

我们重复上面的操作，一直到最后一个字符。

这个过程如下表所示：

当前编码字符	当前目标区间
A	$[0, 0.5)$
A	$[0, 0.25)$
B	$[0.125, 0.225)$
A	$[0.125, 0.175)$
B	$[0.15, 0.17)$
C	$[0.168, 0.17)$
A	$[0.168, 0.169)$
B	$[0.1685, 0.1689)$
A	$[0.1685, 0.1687)$
B	$[0.1686, 0.16868)$

得到最终的目标区间为： $[0.1686, 0.16868)$ ，我们在这个区间内，任意选一个小数，便可以作为最终的编码小数。但是计算机只能识别 0 和 1，所以我们再将小数转成二进制。我们的诉求是进行最短压缩，所以我们要从 $[0.1686, 0.16868)$ 选一个二进制表示最短的小数。这里我们选定 0.16864013671875，二进制为：0.00101011001011，去掉整数位 0 以及小数点后，最终的二进制编码为 00101011001011，bit 长度为 14 位，比哈夫曼编码要更短 1 位。

译码：

理解了编码过程，那么解码过程也就很容易理解了。上面最终二进制编码为 00101011001011，加上小数点后还原为 0.00101011001011，对应的十进制编码小数是 0.16864013671875。

我们先从初始区间中定位第一个字符：

A:[0, 0.5), B:[0.5, 0.9), C:[0.9, 1)

0.16864013671875 位于 A 区间，所以第一个字符为 A。

我们接着对 A:[0, 0.5) 再进行划分。

A:[0, 0.25), B:[0.25, 0.45), C:[0.45, 0.5)

0.16864013671875 仍然位于 A 区间，所以第二个字符仍然为 A。

我们接着对 A:[0, 0.25) 再进行划分。

A:[0, 0.125), B:[0.125, 0.225), C:[0.225, 0.25)

0.16864013671875 位于 B 区间，所以第三个字符为 B。

我们接着对 B:[0.125, 0.225) 再进行划分。

A:[0.125, 0.175), B:[0.175, 0.215), C:[0.215, 0.225)

0.16864013671875 位于 A 区间，所以第四个字符为 A。

依次类推，我们可以从 0.16864013671875 将整个字符解码出来。

以上就是算术编码的编码与译码原理。

2.2 实现原理

编码算法的伪代码如下：

Set low to 0.0

Set high to 1.0

While there are still input symbols do

 get an input symbol

 code_range = high - low.

 high = low + code_range * high_range of the symbol being coded

 low = low + code_range * low_range of the symbol being coded

End of While

output low

译码算法的思想是：找到码坐落区间的对应字符，将该字符输出，然后更新码。伪代码如下：

get the number encoding the data

loop

current symbol = the symbol/character in which range the number falls

current range = current symbols high value – current symbols low value

subtract current symbols low value from number

divide the number by the current range

end loop

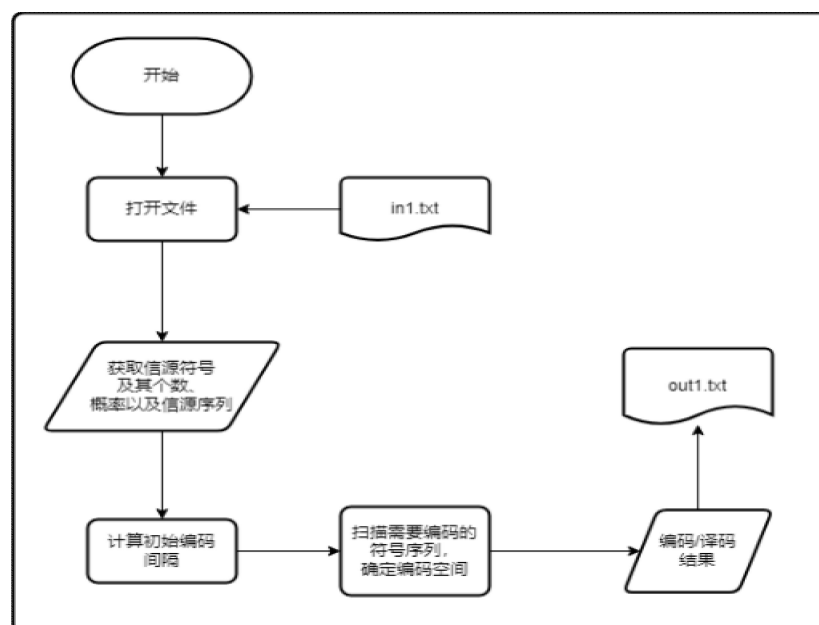
实际编码中还需要解决以下三个问题：

①结束符：实际解码中，最后区间不一定会正好回到 0.0-1.0，因此需要设定一个结束符来代表字符串的结束。（可以默认使用‘#’作为结束符，在读取字符串和编码、解码时，判断到‘#’即停止计算。）

②概率自适应：大多数例子中接收方实际上事先已经知道了字符串中各字符出现的次数。但是这在实际环境中是不现实的，因此要设立一个发送方与接收方都遵守的机制，在事先不沟通编码字符串的统计信息的情况下，能够编码及解码字符串。（本次设计中未实现）

③溢出问题：随着需要编码的字符串长度的增长，最后计算得的小数的位数就会越多。无论采用什么样的编程语言，最后都会面临浮点数溢出的问题。（在实现时，一旦发现 Low 和 High 的有效数字收敛，那么就可以认为这些位的数字对后续计算不会产生影响。可以输出这些已经收敛的数字。同时小数点可以向后移动几位获得新区间，在新区间上继续计算后面的子区间。这样编码区间永远保持在一个有限的精度要求上。）

2.3 流程图



3.实现源码

```
//算术编码与译码
#include<iostream>
#include<iomanip>
#include<fstream>
#include<algorithm>
#include<string.h>
#include<string>
#include<set>
#include<cmath>
#include <sstream>
#include "math.h"

#pragma warning(disable:4996)
using namespace std;

//定义全局变量
char S1[100], S2[100], A[10]; //用户的原数据码字, 保存的输入编码字符集
double P[10], f1[10], f2[10], gFs1, gFs2; //编码字符概率, 映射区间边界点,
char bm[100], jm[100]; //保存码字
double l1, l2; //保存算术编码的码字长度

fstream in("E:/A-Develop/Microsoft Visual
Studio/A-PROGRAM/PROGRAM/CourseDesign/EncodeDecode/in1.txt");
fstream out("E:/A-Develop/Microsoft Visual
Studio/A-PROGRAM/PROGRAM/CourseDesign/EncodeDecode/out1.txt");

void encode(int a, int h1, int h2);
void decode(int a);

void main()
{
    int a, i, h1, h2 = 0; //a 为编码字符集个数, i 为循环变量, h 为原始数据字符串长度
    const char lim = '#';
    string str;
    bool beg = false;

    cout << "编码符号个数: ";
    cin >> a;
    cout << endl;

    cout << "输入编码符号及其概率值: " << endl;
    for (i = 0; i < a; i++)
```

```

{
    cin >> A[i] >> P[i];
}

//将要编码的数据映射到一个位于[0,1)的实数区间中，即划分初始区间
for (i = 1; i < a; i++)
{
    f1[0] = 0; f2[0] = 0;
    f1[i] = f1[i - 1] + P[i - 1];
    f2[i] = f2[i - 1] + P[i - 1];
}

//进行算术编码与译码
while (!in.eof())
{
    cout << "输入需要编码的符号序列，同时用#结尾：" << endl;
    h1 = 0; h2 = 0;

    //将第一组要编码的字符存入到数组 S1 中
    while (1)
    {
        char cc;
        in >> cc;
        if (cc == '#')
            break;//在以“#”为结尾的时候结束存储
        S1[h1++] = cc;
        cout << cc;
    }
    cout << endl;

    //将第二组要编码的字符存入到数组 S2 中
    while (1)
    {
        char cc2;
        in >> cc2;
        if (cc2 == '#')
            break;//在以“#”为结尾的时候结束存储
        S2[h2++] = cc2;
        cout << cc2;
    }
    cout << endl;

    cout << endl;
}

```

```

        cout << "算术编码结果: " << endl;
        encode(a, h1, h2);

        cout << "对应的译码结果: " << endl;
        decode(a);

        system("pause");
    }
    in.close();
    out.close();
}

void encode(int a, int h1, int h2)//算术编码, a 为编码字符集个数, h 为原始数据字符串长度
{
    int i, j;    //循环变量
    double fr;    //划分区间的对应点
    double D1 = 1, D2 = 1;    //区间间隔
    double Fs1 = 0, Fs2 = 0;    //区间起始点
    double Sp1[100], Sp2[100];    //S 数组中字符对应的概率

    //以待编码的个数和字符集个数为循环周期, 将待编码的字符串所对应的概率小数存入到 Fs 中
    for (i = 0; i < h1; i++)
    {
        for (j = 0; j < a; j++)
        {
            if (S1[i] == A[j])
            {
                Sp1[i] = P[j];
                fr = f1[j];    //将划分好的[0, 1)区间的对应起始点赋值给 fr
            }
        }
        Fs1 = Fs1 + D1 * fr;
        D1 *= Sp1[i];
    }
    cout << "Fs1=" << fixed << setprecision(20) << Fs1 << endl;

    for (i = 0; i < h2; i++)
    {
        for (j = 0; j < a; j++)
        {
            if (S2[i] == A[j])
            {

```

```

        Sp2[i] = P[j];
        fr = f2[j]; //将划分好的[0,1)区间的对应起始点赋值给 fr
    }
}
Fs2 = Fs2 + D2 * fr;
D2 *= Sp2[i];
}
cout << "Fs2=" << fixed << setprecision(20) << Fs2 << endl;

gFs1 = Fs1;
gFs2 = Fs2;

out << "算术编码结果为: " << endl;
out << "Fs1=" << fixed << setprecision(16) << Fs1 << endl;
out << "Fs2=" << fixed << setprecision(16) << Fs2 << endl;


l1 = log((double)1 / D1) / log((double)2); //计算算术编码的码字长度 l1
if (l1 > (int)l1)
    l1 = (int)l1 + 1;
else
    l1 = int(l1);

//将 Fs 转换成二进制形式(乘二取个位)
int d1[100];
int m1 = 0; //表示二进制形式位数
while (l1 > m1)
{
    Fs1 = 2 * Fs1;
    if (Fs1 > 1)
    {
        Fs1 = Fs1 - 1;
        d1[m1] = 1;
    }
    else if (Fs1 < 1)
        d1[m1] = 0;
    else
    {
        d1[m1] = 1;
        break;
    }
    m1++;
}
}

```



```

int z = m1;

/*if (m1 >= 11)      //解决有关算术编码的进位问题
{
    while (1)
    {
        d1[m1 - 1] = (d1[m1 - 1] + 1) % 2;
        if (d1[m1 - 1] == 1) break;
        else m1--;
    }
}*/

cout << "s1=";
out << "s1=";

for (int e = 0; e < z; e++)
{
    cout << d1[e];
    out << d1[e];
}
cout << endl;
out << endl;


l2 = log((double)1 / D2) / log((double)2);    //计算算术编码的码字长度 l2
if (l2 > (int)l2)
    l2 = (int)l2 + 1;
else
    l2 = int(l2);

//将 Fs 转换成二进制形式(乘二取个位)
int d2[100];
int m2 = 0;
while (l2 > m2)
{
    Fs2 = 2 * Fs2;
    if (Fs2 > 1)
    {
        Fs2 = Fs2 - 1;
        d2[m2] = 1;
    }
    else if (Fs2 < 1)

```

```

        d2[m2] = 0;
    else
    {
        d2[m2] = 1;
        break;
    }
    m2++;
}

int z2 = m2;

/*if (m2 >= 12)    //解决有关算术编码的进位问题
{
    while (1)
    {
        d2[m2 - 1] = (d2[m2 - 1] + 1) % 2;
        if (d2[m2 - 1] == 1) break;
        else m2--;
    }
}*/

cout << "s2=";
out << "s2=";

for (int e = 0; e < z2; e++)
{
    cout << d2[e];
    out << d2[e];
}

cout << endl;
out << endl;
}

```

```

void decode(int a)    //算术译码, a 为编码字符集个数
{
    int i = 0;
    double Ft1, Pt1, Ft2, Pt2;
    double Fs1 = 0, Fs2 = 0, D1 = 1, D2 = 1;
    double L1, L2; //保存由编码结果预计的原数据长度

    std::string str1;

```

```

std::stringstream ssl;
ssl << std::setprecision(21) << gFs1;
str1 = ssl.str();

std::string str2;
std::stringstream ss2;
ss2 << std::setprecision(21) << gFs2;
str2 = ss2.str();

out << "算术译码后: \n";
while (1)
{
    for (int j = a - 1; j > -1; j--)
    {
        Ft1 = Fs1;
        Pt1 = D1;
        Ft1 += Pt1 * f1[j];    //对其进行逆编码
        Pt1 *= P[j];
        if (gFs1 >= Ft1)      //对其进行判断并将值存入到数组 A 中
        {
            Fs1 = Ft1;
            D1 = Pt1;
            cout << A[j];
            out << A[j];
            break;
        }
    }
    i++;

    L1 = log((double)1 / D1) / log((double)2);
    if (L1 > (int)L1)
        L1 = (int)L1 + 1;
    else
        L1 = int(L1);

    if (L1-4 > str1.length())
        break;
}

/*for (i = 0; i < h1; i++)    //以编码个数和符号个数为循环周期进行解码
{
    for (int j = a - 1; j > -1; j--)
    {
        Ft1 = Fs1;

```

```

        Pt1 = D1;
        Ft1 += Pt1 * f1[j];    //对其进行逆编码
        Pt1 *= P[j];
        if (gFs1 >= Ft1)      //对其进行判断并将值存入到数组 A 中
        {
            Fs1 = Ft1;
            D1 = Pt1;
            cout << A[j];
            out << A[j];
            break;
        }
    }
}*/

cout << endl;
out << endl;

i = 0;
while (1)
{
    for (int j = a - 1; j > -1; j--)
    {
        Ft2 = Fs2;
        Pt2 = D2;
        Ft2 += Pt2 * f2[j];    //对其进行逆编码
        Pt2 *= P[j];
        if (gFs2 >= Ft2)      //对其进行判断并将值存入到数组 A 中
        {
            Fs2 = Ft2;
            D2 = Pt2;
            cout << A[j];
            out << A[j];
            break;
        }
    }
    i++;
    L2 = log((double)1 / D2) / log((double)2);
    if (L2 > (int)L2)
        L2 = (int)L2 + 1;
    else
        L2 = int(L2);

    if (L2-4 > str2.length())
        break;
}

```



```
out1.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
算术编码结果为:
Fs1=0.1615720499279920
Fs2=0.3711790336183600
s1=00101001010111001100100100
s2=01011111000001011001011011
算术译码后:
abbbabbbabbbabbbabbbabbbabbbabbb
babbbabbbabbbabbbabbbabbbabbbabbb

第 1 行, 第 1 列 100% Windows (CRLF) ANSI
```

假使接收方不知道原始数据长度，更改译码循环条件后：

```
out1.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
算术编码结果为:
Fs1=0.1615720499279920
Fs2=0.3711790336183600
s1=00101001010111001100100100
s2=01011111000001011001011011
算术译码后:
abbbabbbabbbabbbabbbabbbabbbabbb
babbbabbbabbbabbbabbbabbbabbbabbb

第 9 行, 第 1 列 100% Windows (CRLF) ANSI
```

4.3 终端运行界面

```
C:\> E:\A-Develop\Microsoft Visual Studio\A-PROGRAM\PROGRAM\Cour
编码符号个数：2
输入编码符号及其概率值：
a 0.25 b 0.75
输入需要编码的符号序列，同时用#结尾：
abbbabbbabbbabbbabbbabbbabbbabbb
babbbabbbabbbabbbabbbabbbabbbabbb

算术编码结果：
Fs1=0.1615720499279920
Fs2=0.3711790336183600
s1=00101001010111001100100100
s2=01011111000001011001011011
对应的译码结果：
abbbabbbabbbabbbabbbabbbabbbabbb
babbbabbbabbbabbbabbbabbbabbbabbb
请按任意键继续. . .
```

假使接收方不知道原始数据长度，更改译码循环条件后：

```
E:\A-Develop\Microsoft Visual Studio\A-PROGRAM\PROC
编码符号个数: 2
输入编码符号及其概率值:
a 0.25 b 0.75
输入需要编码的符号序列, 同时用#结尾:
abbbabbbabbbabbbabbbabbbabbbabbb
babbbabbbabbbabbbabbbabbbabbbabb

算术编码结果:
Fs1=0.16157204992799201193
Fs2=0.37117903361836002141
s1=00101001010111001100100100
s2=01011111000001011001011011
对应的译码结果:
abbbabbbabbbabbbabbbabbbabbbabbb
babbbabbbabbbabbbabbbabbbabbbabba
请按任意键继续. . .
```

运行结果符合预期, 各项功能均正确, 编码译码功能正常。

4.4 结果分析

本次设计中, 编码符号个数、编码符号及其概率值需要用户自行输入; 输入端输入的两串符号序列从文件 `in1.txt` 中获得; 两串符号序列的算术编码的结果输出为小数值及其二进制串; 假使接收端不知道原始数据长度进行译码, 输出对应的译码结果。

经过分析可得, 该输出端的二元无记忆信源的平均熵为 $H(s) = 0.8113$, 两串符号序列长度 L 均为 32, $P(s) = \frac{3^{24}}{4^{32}}$, 编码长度 k 经计算 $k = \left\lceil \log_2 \frac{1}{P(s)} \right\rceil = 26$, 所以编码效率为 $\eta = \frac{H(s)}{k/L} = 99.85\%$ 。

可改进的地方:

1. 可以采用自适应算术编码实时调整字符出现概率, 或者采用完全统计模型算术编码对已经输入的所有字符进行概率统计, 然后再进行编码和译码。
2. 译码循环判断条件可以再进行优化一下。
3. 编码译码过程中溢出问题的处理可以再优化。

二、 综合编码 1:

1.任务说明

编码：游程编码+Huffman 编码+加密编码+信道编码及对应的译码

一个二元无记忆信源，0 符号的出现概率为 $1/4$, 1 符号的出现概率为 $3/4$ 。现要求对该信源连续出现的 n 个符号序列，进行游程编码/对游程编码结果进行 Huffman 编码（可用教材的 MH 编码方案）/使用加密算法进行加密/进行信道编码；然后模拟信道的传输过程，并对收到的信息串进行信道译码/解密译码/Huffman 译码/游程译码。

假定，连续出现的 0 或 1 序列的长度不超过 16， n 不小于 256。

其中加密编码/解密译码可自主选择对称加密算法或非对称算法；

信道编码要求采用(7,4)系统循环码,其中, $g(x)=x^3+x+1$ ，译码采用简化的译码表；

信道为 BSC 信道， $p=10^{-2}$

输入：长为 n 的 0/1 串

输出：1. 游程编码结果，2. Huffman 编码结果，3. 加密编码结果 4. 信道编码结果 5. 模拟接收串，6. 信道译码结果，7. 解密编码结果 8. Huffman 译码结果 9. 游程译码结果

输入文件:in3.txt，一组输入即可

输出文件:out3.txt，对输入的处理结果

2.问题分析、实现原理、流程图

2.1 问题分析

① 游程编码与译码

游程编码是一种比较简单的压缩算法，其基本思想是将重复且连续出现多次的字符使用（连续出现次数，某个字符）来描述。

比如一个字符串：AAAAABBBBCCCC

使用游程编码可以将其描述为：5A4B3C

5A 表示这个地方有 5 个连续的 A，同理 4B 表示有 4 个连续的 B，3C 表示有 3 个连续的 C，其它情况以此类推。

在对图像数据进行编码时，沿一定方向排列的具有相同灰度值的像素可看成是连续符号，用字符串代替这些连续符号，可大幅度减少数据量。游程编码记录方式有两种：①逐行记录每个游程的终点列号；②逐行记录每个游程的长度（像素数）。

译码逆推即可。

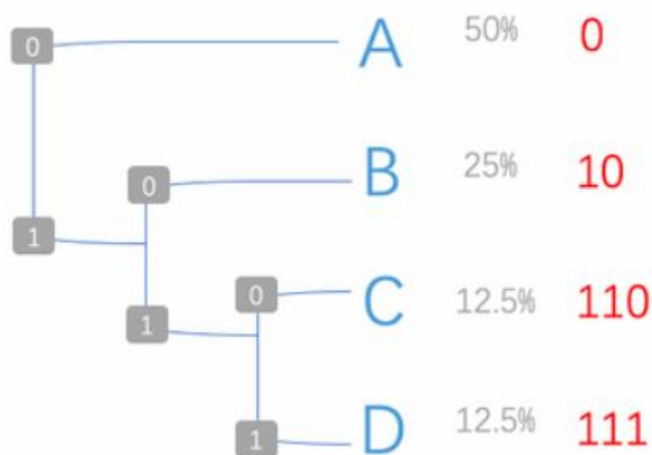
② Huffman 编码与译码

霍夫曼编码是一种无损的统计编码方法，利用信息符号概率分布特性来改编字长进行编码。适用于多元独立信源。霍夫曼编码对于出现概率大的信息符号用字长小的符号表示，对于出现概率小的信息用字长大的符号代替。如果码字长严格按照所对应符号出现的概率大小逆序排列，则编码结果的平均字长一定小于其他排列形式。

霍夫曼编码的具体步骤如下：

- 1) 将信源符号的概率按减小的顺序排队。
- 2) 把两个最小的概率相加，并继续这一步骤，始终将较高的概率分支放在右边，直到最后变成概率。
- 3) 画出由概率 1 处到每个信源符号的路径，顺序记下沿路径的 0 和 1，所得就是该符号的霍夫曼码字。
- 4) 将每对组合的左边一个指定为 0，右边一个指定为 1（或相反）。

如下图例子：



译码逆推即可。

③ 加密编码与译码

对称加密算法是应用较早的加密算法，又称为共享密钥加密算法。在对称加密算法中，使用的密钥只有一个，发送和接收双方都使用这个密钥对数据进行加密和解密。这就要求加密和解密方事先都必须知道加密的密钥。

数据加密过程：在对称加密算法中，数据发送方将明文(原始数据)和加密密钥一起经过特殊加密处理，生成复杂的加密密文进行发送。

数据解密过程：数据接收方收到密文后，若想读取原数据，则需要使用加密使用的密钥及相同算法的逆算法对加密的密文进行解密，才能使其恢复成可读明文。

④ 信道编码与译码

本次设计中的信道编码要求采用(7,4)系统循环码,其中, $g(x) = x^3 + x + 1$ ，译码采用简化的译码表。

在编码时，首先需要根据给定循环码的参数确定生成多项式 $g(x)$ ，也就是从 $F(x)$ 的因子中选一个 $(n-k)$ 次多项式作为 $g(x)$ ；然后，利用循环码的编码特点，即所有循环码多项式 $A(x)$ 都可以被 $g(x)$ 整除，来定义生成多项式 $g(x)$ 。据上述原理可以得到一个较简单的系统：设要产生 (n,k) 循环码， $m(x)$ 表示信息多项式，循环码编码方法，则其次数必小于 k ，而 $m(x)$ 的次数必小于 n ，用 $m(x)$ 除以 $g(x)$ ，

可得余数 $r(x)$ ， $r(x)$ 的次数必小于 $(n-k)$ ，将 $r(x)$ 加到信息位后作监督位，就得到了系统循环码。

具体解释即：

(1) 确定 $g(x)$ ，用 x^{n-k} 乘 $m(x)$ 。这一运算实际上是把信息码后附加上 $(n-k)$ 个“0”。

例如信息码为 1100，写成多项式为 $m(x)=x^3+x^2$ ，当 $n-k=7-4=3$ 时， $x^{n-k} \cdot m(x)=x^6+x^5$ ，表示码组 1100000。

(2) 求 $r(x)$ 。由于循环码多项式 $A(x)$ 都可以被 $g(x)$ 整除，因此，用 $x^{n-k} \cdot m(x)$ 除以 $g(x)$ ，就得到商 $Q(x)$ 和余式 $r(x)$ ，这样就得到了 $r(x)$ 。

即

$m(x)]/[g(x)]=Q(x)+r(x)/g(x)=(x^6+x^5)/(x^3+x^2+1)=(x^3+1)+(x^2+1)/(x^3+x^2+1)$ ，即和下面式子等效 $1100000/1101=1001+101/1101$ 。

(3) 编码输出系统循环码多项式 $A(x)$ 为： $A(x)=x^{n-k} \cdot m(x) + r(x)$ 。这时的编码输出为： $1100000+101=1100101$ 。

对于接收端译码的要求通常有两个：检错与纠错。达到检错目的的译码非常简单通过判断接收到的码组多项式 $B(x)$ 是否能被生成多项式 $g(x)$ 整除作为依据。当传输中未发生错误时，也就是接收的码组与发送的码组相同，即 $A(x)=B(x)$ ，则接收的码组 $B(x)$ 必能被 $g(x)$ 整除；若传输中发生了错误，则 $A(x) \neq B(x)$ ， $B(x)$ 不能被 $g(x)$ 整除。因此，可以根据余项是否为零来判断码组中是否有错码。需要指出的是，有错码的接收码组也有可能被 $g(x)$ 整除，这时的错码就不能检出了。这种错误被称为不可检错误，不可检错误中的错码数必将超过这种编码的检错能力。在接收端为纠错而采用的译码方法自然比检错要复杂许多，因此，对纠错码的研究大都集中在译码算法上。

此处可以在译码时采用查表的方法。

⑤ 信道传输

使用二元对称无记忆离散信道，信道为 BSC 信道， $p=10^{-2}$ 。

2.2 实现原理

① 游程编码与译码

对于二元相关信源，其输出只有两个符号，即"0"和"1"，"0"游程和"1"游程总是交替出现。我们就只需对串的长度（即游程长度）进行标记，这只需用一个表示串的长度的标志参量就行。然后就可将信源输出的任意一维二元序列一一对应地映射成交替出现的游程长度的标志序列。当然一般游程长度都用自然数标记，所以就映射成交替出现的表示游程长度的自然数序列，简称游程序列。这种映射是可逆的，是无失真的。例如某二元序列为：0010011111100000001... 可映射成游程序列：02 11 02 16 07 11...

译码时根据原理逆推即可。

② Huffman 编码与译码

二元霍夫曼编码的基本步骤如下：

- 1) 将信源符号的概率按减小的顺序排队。
- 2) 用 0 和 1 码符号分别分配给概率最小的两个信源符号，并将这两个概率最小的信源符号合并成一个新符号，并用这两个最小概率之和作为新符号的概率，从而得到只包含 $q-1$ 个符号的新信源，称为 S 信源的缩减信源 S_1 。
- 3) 把缩减信源 S_1 的符号仍按概率大小以递减次序排列，再将其最后两个概率最小的符号合并成一个新符号，并分别用 0 和 1 码符号表示，这样又形成了 $q-2$ 个符号的缩减信源 S_2 。
- 4) 依次继续下去，直至缩减信源最后只剩两个符号为止。将这最后两个符号分别用 0 和 1 码符号表示。最后这两个符号的概率之和必为 1。然后从最后一级缩减信源开始，依编码路径由后向前返回，就得出各信源符号所对应的码符号序列，即得对应的码字。

译码时同理逆推即可。

③ 加密编码与译码

对称加密算法本次实验选择的是仿射密码。仿射密码为单表加密的一种，字母系统中所有字母都藉一简单数学方程加密，对应至数值，或转回字母。

仿射密码是一种替换密码。它是一个字母对一个字母的。它的加密函数是 $e(x) = ax + b \pmod{m}$ ，其中 a 和 m 互质， m 是字母的数目。解码函数是 $d(x) = a^{-1}(x - b) \pmod{m}$ ，其中 a^{-1} 是 a 在 Z_m 群的乘法逆元。

仿射密码为单表加密的一种，字母系统中所有字母都藉一简单数学方程加密，对应至数值，或转回字母。 其仍有所有替代密码之弱处。所有字母皆借由方程 $(ax + b) \pmod{26}$ 加密， b 为移动大小。

④ 信道编码与译码

本次设计中的信道采用（7，4）系统循环码，又叫（7，4，3）循环汉明码，共有 16 个许用码字。

其标准生成矩阵为：

$$G(x) = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

其一致监督矩阵为：

$$H(x) = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

其(7,4)码表为：

序号	输入序列	输出序列	序号	输入序列	输出序列
1	0000	0000000	9	1000	1000110
2	0001	0001101	10	1001	1001011
3	0010	0010111	11	1010	1010001
4	0011	0011010	12	1011	1011100
5	0100	0100011	13	1100	1100101
6	0101	0101110	14	1101	1101000
7	0110	0110100	15	1110	1110010
8	0111	0111001	16	1111	1111111

编码时，在算法实现时，采用信息位计算监督位，然后将两者组合的方法，用四个信息位计算出三个监督位之后，将三个监督位拼接在四个信息位的后面，译码时采用查表的方法。

其(7,4)码译码表为：

序号	输入序列	输出序列
1	0000000 及其 1 位出错码组	0000
2	0001101 及其 1 位出错码组	0001
3	0010111 及其 1 位出错码组	0010
4	0011010 及其 1 位出错码组	0011
5	0100011 及其 1 位出错码组	0100
6	0101110 及其 1 位出错码组	0101
7	0110100 及其 1 位出错码组	0110
8	0111001 及其 1 位出错码组	0111
9	1000110 及其 1 位出错码组	1000
10	1001011 及其 1 位出错码组	1001
11	1010001 及其 1 位出错码组	1010
12	1011100 及其 1 位出错码组	1011
13	1100101 及其 1 位出错码组	1100
14	1101000 及其 1 位出错码组	1101
15	1110010 及其 1 位出错码组	1110
16	1111111 及其 1 位出错码组	1111

⑤ 信道传输

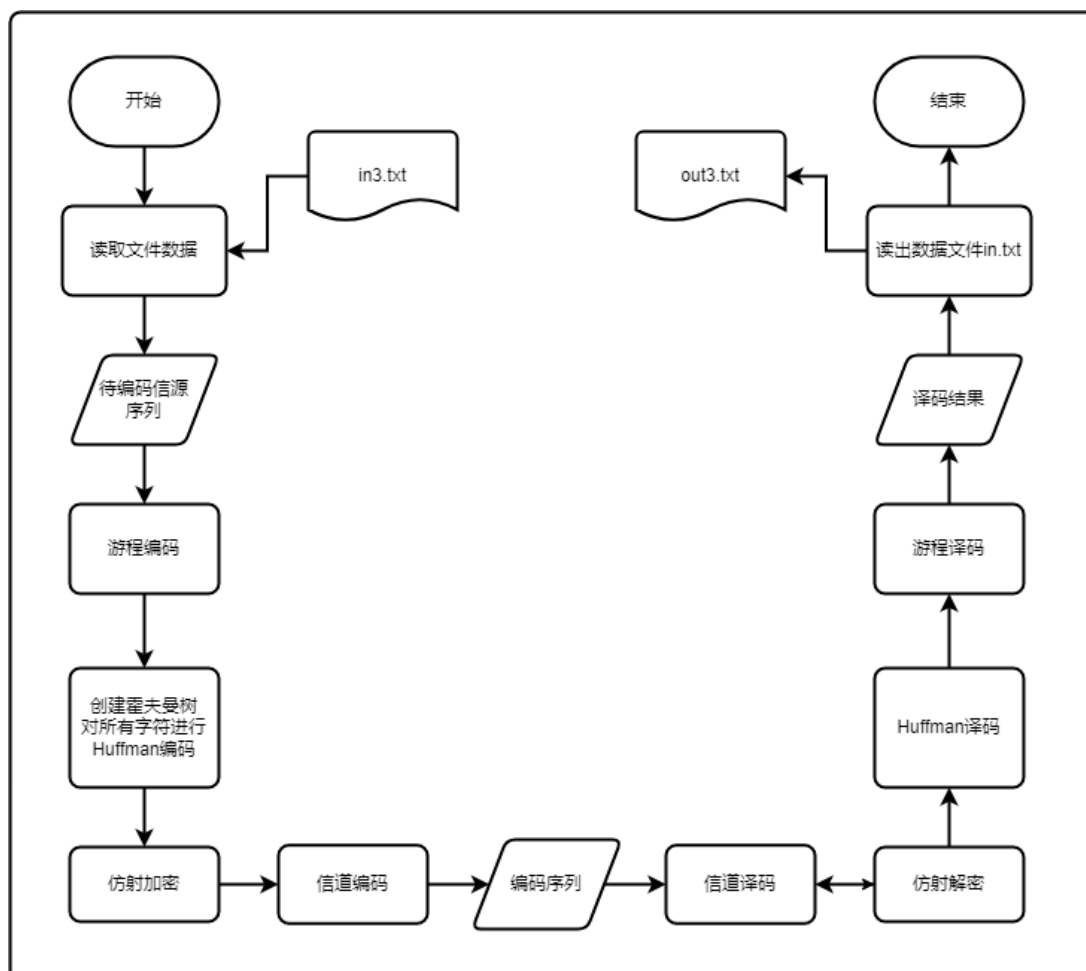
使用二元对称无记忆离散信道，信道为 BSC 信道， $p=10^{-2}$ 。

二元对称信道信道的转移概率为（ p 表示错误概率）

$$P(j|k) = \begin{cases} 1 - p, & k = j \\ p, & k \neq j, j = 0, 1, \dots \end{cases}$$

C++中常用 `rand()`函数生成随机数，但严格意义上来讲生成的只是伪随机数（pseudo-random integral number）。生成随机数时需要我们指定一个种子，如果在程序内循环，那么下一次生成随机数时调用上一次的结果作为种子。但如果分两次执行程序，那么由于种子相同，生成的“随机数”也是相同的。`srand((unsigned)time(NULL))`则使用系统定时/计数器的值作为随机种子。每个种子对应一组根据算法预先生成的随机数，所以，在相同的平台环境下，不同时间产生的随机数会是不同的，相应的，若将 `srand((unsigned)time(NULL))`改为 `srand(TP)`（TP 为任一常量），则无论何时运行、运行多少次得到的“随机数”都会是一组固定的序列，因此 `srand` 生成的随机数是伪随机数。我们通常用 `srand((unsigned)time(0))`或者 `srand((unsigned)time(NULL))`来产生种子。如果仍然觉得时间间隔太小，可以在 `(unsigned)time(0)`或者 `(unsigned)time(NULL)`后面乘上某个合适的整数。

2.3 流程图



3.实现源码

```
//综合编码与译码 1
#include<iostream>
#include<string>
#include<stdio.h>
#include<stdlib.h>
#include<fstream>
#include<cctype>
#include<algorithm>
#include<cmath>
#include<vector>
#include<ctime>

#pragma warning(disable:4996)
#define N 200
#define M 2*N-1

#define TOLOWER(p) {transform(p.begin(),p.end(),p.begin(),::tolower);}

typedef char* HuffmanCode[2 * M];    //huffman 编码

//typedef long long LL;
using namespace std;

char x[1024];    //游程编码
char y[1024];    //huffman 编码

fstream in("E:/A-Develop/Microsoft Visual
Studio/A-PROGRAM/PROGRAM/CourseDesign/EncodeDecode/in3.txt");
fstream out("E:/A-Develop/Microsoft Visual
Studio/A-PROGRAM/PROGRAM/CourseDesign/EncodeDecode/out3.txt");

void Runlength_encode(char a[1024]);    //游程编码函数
void Runlength_decode(char a[1024]);    //游程译码函数
void Huffman_encode(char* ch);    //Huffman 编码函数
void Huffman_decode();    //Huffman 译码函数
int canshu(int k, int b);    //判断仿射加密参数是否合法
string FSjiami(string str, int k, int b);    //仿射加密函数
string FSjiemi(string pass, int k, int b);    //仿射解密函数
void create_G();    //初始化标准生成矩阵
void create_S();    //初始化译码表
```



```

int bin_to_deci(string inin);    //二进制转十进制
string deciToBin_8bit(int deci);    //十进制转八位二进制
void string_to_ascii(string info, string a[]);    //转换成 ASCII 码串
string channel_encode(string infomation);    //信道编码
string channel_decode(string infomation);    //信道译码
string BSC(string infomation);    //模拟 BSC 信道传输

int main()
{
    const char lim = '#';
    char s1[1024];

    int k, b;    //加密算法的参数
    cout << "请输入加密算法的参数(k,b): ";
    cin >> k >> b;
    while (!canshu(k, b))
    {
        cout << "参数不合法（参数 k 应该与 mod * 26 互为素数），请重新输入：";
        cin >> k >> b;
    }

    if (!in.is_open())
    {
        cout << "Error opening file"; exit(1);
    }

    while (!in.eof())
    {
        in.getline(s1, 1024, lim);
        if (s1[0] == '\0' || s1[0] == '\n')
            break;
        cout << "信源的符号序列: " << s1 << endl;
        out << "信源的符号序列: " << s1 << endl << endl;
        cout << endl;

        for (int q = 0; q < 1024; q++)
        {
            x[q] = '\0';
            y[q] = '\0';
        }

        Runlength_encode(s1);
    }
}

```

```

//对 s1 进行游程编码得到 x 字符串数组
cout << endl;
out << endl;

Huffman_encode(x);
//对 x 进行 Huffman 编码得到 y 字符串数组
cout << endl;
out << endl;

string jiamil;    //加密输出密文 (ASCII 码串)
jiamil = FSjiami(y, k, b);
cout << "仿射加密后: " << jiamil << endl;
out << "仿射加密后: " << jiamil << endl;
cout << endl;
out << endl;

create_G();    //生成矩阵
create_S();    //生成译码表
int lenth = jiamil.length();
string* a = new string[lenth];    //存储密文的 ASCII 码串
string* output = new string[lenth];    //存储信道编码过程的串
string_to_ascii(jiamil, a);    //密文转化为 ASCII 码串
string output1 = "";    //存储信道译码后的 ASCII 码串密文

cout << "信道编码: ";
out << "信道编码: ";
for (int i = 0; i < lenth; i++)
{
    output[i] = channel_encode(a[i]);    //信道编码
    cout << output[i];
    out << output[i];
}
cout << endl << endl;
out << endl << endl;

cout << "模拟传递串: ";
out << "模拟传递串: ";
for (int i = 0; i < lenth; i++)
{
    output[i] = BSC(output[i]);    //信道传输
    cout << output[i];
    out << output[i];
}

```

```

        cout << endl << endl;
        out << endl << endl;

        cout << "信道译码: ";
        out << "信道译码: ";
        for (int i = 0; i < lenth; i++)
        {
            output[i] = channel_decode(output[i]);    //信道译码
            cout << output[i];
            out << output[i];
            int f = 0;
            f = bin_to_deci(output[i]); //将二进制转成十进制
            output1 += (char)f; //变成 ASCII 码对应的字符，即加密后的密文

        }
        cout << endl << endl;
        out << endl << endl;

        //仿射解密后的字符串
        string jiemil = FSjiemil(output1, k, b);
        cout << "仿射解密后: " << jiemil << endl;
        out << "仿射解密后: " << jiemil << endl;
        cout << endl;
        out << endl;

        //Huffman 译码
        Huffman_decode();
        cout << endl;
        out << endl;

        //游程译码
        Runlength_decode(x);
        //对 x 进行游程译码
        cout << endl;
        out << endl;

    }
    in.close();
    out.close();
    return 0;
}

//游程编码

```

```

void Runlength_encode(char a[1024])
{
    char yc[1024];
    int i = 0, j = 0, count = 1;
    yc[0] = a[0];    //游程编码第一个字符为字符串第一个字符
    for (i = 0; a[i] != '\0'; i++)    //遍历字符串
    {
        if (a[i] == a[i + 1])    //统计相同字符数量
            count++;
        else
        {
            yc[j + 1] = count + 48;    //ASCII 码把 int 转换成字符 char
            j = j + 2;
            yc[j] = a[i + 1];
            count = 1;
        }
    }
    yc[j] = '\0';
    cout << "游程编码后: " << yc << endl;
    out << "游程编码后: " << yc << endl;
    strcpy_s(x, yc);
}

```

//游程译码

```

void Runlength_decode(char a[1024])
{
    char ym[1024];
    int j = 0, count, sign = 0;
    for (int i = 0; a[i] != '\0'; i = i + 2)
    {
        ym[sign] = a[i];    //第一位为标志位表示游程符号
        for (j = sign, count = 1; count <= a[i + 1] - 48; count++, j++)
            ym[j] = a[i];    //遍历相同数量的符号装入译码字符串中
        sign = j;
    }
    ym[j] = '\0';
    cout << "游程译码后: " << ym << endl;
    out << "游程译码后: " << ym << endl;
}

```

//Huffman 树的数据结构定义

```

typedef struct
{
    int weight;    //权值

```

```

    int parent;    //父节点
    int LChild;    //左子节点
    int RChild;    //右子节点
}HTNode, Huffman[M + 1];    //Huffman 树

typedef struct
{
    int Weight;    //叶子节点的权值
    char c;    //叶子节点
    int num;    //叶子节点的二进制码的长度
}WNode, WeightNode[N];

//产生叶子节点的字符和权值
void CreateWeight(char ch[], int* s, WeightNode CW, int* p)
{
    int i, j, k;
    int tag;
    *p = 0;    //叶子节点个数

    //统计字符出现个数，放入CW
    for (i = 0; ch[i] != '\0'; i++)
    {
        tag = 1;
        for (j = 0; j < i; j++)
        {
            if (ch[j] == ch[i])
            {
                tag = 0;
                break;
            }
        }
        if (tag)
        {
            CW[++ * p].c = ch[i];
            CW[*p].Weight = 1;
            for (k = i + 1; ch[k] != '\0'; k++)
            {
                if (ch[i] == ch[k])
                    CW[*p].Weight++;    //权值累加
            }
        }
    }
    *s = i;    //字符串长度
}

```

```

//创建 Huffman 树
void CreateHuffmanTree(Huffman ht, WeightNode w, int n)
{
    int i, j;
    int s1, s2;
    //初始化哈夫曼树
    for (i = 1; i <= n; i++)
    {
        ht[i].weight = w[i].Weight;
        ht[i].parent = 0;
        ht[i].LChild = 0;
        ht[i].RChild = 0;
    }
    for (i = n + 1; i <= 2 * n - 1; i++)
    {
        ht[i].weight = 0;
        ht[i].parent = 0;
        ht[i].LChild = 0;
        ht[i].RChild = 0;
    }
    for (i = n + 1; i <= 2 * n - 1; i++)
    {
        for (j = 1; j <= i - 1; j++)
            if (!ht[j].parent)
                break;
        s1 = j; //找到第一个双亲为零的结点
        for (; j <= i - 1; j++)
            if (!ht[j].parent)
                s1 = ht[s1].weight > ht[j].weight ? j : s1;
        ht[s1].parent = i;
        ht[i].LChild = s1;

        for (j = 1; j <= i - 1; j++)
            if (!ht[j].parent)
                break;
        s2 = j; //找到第二个双亲为零的结点
        for (; j <= i - 1; j++)
            if (!ht[j].parent)
                s2 = ht[s2].weight > ht[j].weight ? j : s2;
        ht[s2].parent = i;
        ht[i].RChild = s2;

        ht[i].weight = ht[s1].weight + ht[s2].weight; //权值累加
    }
}

```

```

    }
}

//对叶子结点编码
void CrtHuffmanNodeCode(Huffman ht, char ch[], HuffmanCode h, WeightNode w, int m, int n)
{
    int i, c, p, start;
    char* cd;
    cd = (char*)malloc(n * sizeof(char));
    cd[n - 1] = '\0';//置于末尾
    for (i = 1; i <= n; i++)
    {
        start = n - 1; //cd 串每次从末尾开始
        c = i;
        p = ht[i].parent;//p 在 n+1 至 n-1
        while (p) //沿父亲方向遍历，直到为空
        {
            start--;//依次向前置值
            if (ht[p].LChild == c)//与左子相同,置 0
                cd[start] = '0';
            else//否则置 1
                cd[start] = '1';
            c = p;
            p = ht[p].parent;
        }
        w[i].num = n - start; //二进制码的长度(包含末尾)
        h[i] = (char*)malloc((n - start) * sizeof(char));
        strcpy(h[i], &cd[start]); //将二进制字符串拷贝到指针数组 h 中
    }
    free(cd); //释放 cd 内存
    //system("pause");
}

```

```

//对所有字符的编码
void CrtHuffmanCode(char ch[], HuffmanCode h, HuffmanCode hc, WeightNode w, int n, int m)
{
    int i, k;
    for (i = 0; i < m; i++)
    {
        for (k = 1; k <= n; k++) //从 w[k].c 中查找与 ch[i] 相等的下标 K
            if (ch[i] == w[k].c)
                break;
        hc[i] = (char*)malloc((w[k].num) * sizeof(char));
        strcpy(hc[i], h[k]); //拷贝二进制编码
    }
}

```

```

    }
}

//霍夫曼树解码
void TrsHuffmanTree(Huffman ht, WeightNode w, HuffmanCode hc, int n, int m)
{
    int i = 0, j, p;
    while (i < m)
    {
        p = 2 * n - 1; //从父亲节点向下遍历直到叶子节点
        for (j = 0; hc[i][j] != '\0'; j++)
        {
            if (hc[i][j] == '0')
                p = ht[p].LChild;
            else
                p = ht[p].RChild;
        }
        printf("%c", w[p].c); //打印原信息
        out << w[p].c;
        i++;
    }
    out << endl;
}

```

```

//当霍夫曼树使用完成之后，释放 huffman 编码内存
void FreeHuffmanCode(HuffmanCode h, HuffmanCode hc, int n, int m)
{
    int i;
    for (i = 1; i <= n; i++) //释放叶子结点的编码
        free(h[i]);
    for (i = 0; i < m; i++) //释放所有结点的编码
        free(hc[i]);
}

```

```

int n;    //n 为叶子结点的个数
int m;    //m 为字符串 ch[] 的长度
Huffman ht;    //Huffman 二叉树
HuffmanCode h, hc;    //h 存放叶子结点的编码，hc 存放所有结点的编码
WeightNode w;    //存放叶子节点的信息

```

```

//Huffman 编码函数
void Huffman_encode(char* ch)
{
    n = 0;

```



```

int i;
m = 0;
CreateWeight(ch, &m, w, &n);    //产生叶子结点信息 w, m 为字符串 ch[] 的长度, n 为叶子结点的个数

CreateHuffmanTree(ht, w, n);    //产生 Huffman 树 ht
CrtHuffmanNodeCode(ht, ch, h, w, m, n);    //叶子结点的编码 h
CrtHuffmanCode(ch, h, hc, w, n, m);    //所有字符的编码 hc
cout << "Huffman 编码后: ";    //打印字符串的编码
out << "Huffman 编码后: ";
int j = 0;
for (i = 0; i < m; i++)
{
    cout << hc[i];
    out << hc[i];
    if (y[i] == '\0')
        strcpy(&y[i], hc[i]);
    else if (y[i] != '\0')
    {
        while (y[j] != '\0')
            j++;
        strcpy(&y[j], hc[i]);
    }
}
cout << endl;
out << endl;
//system("pause");
}

//Huffman 译码函数
void Huffman_decode()
{
    cout << "huffman 译码后: ";
    out << "huffman 译码后: ";
    TrsHuffmanTree(ht, w, hc, n, m);    //译码, 打印 w[p].c
    FreeHuffmanCode(h, hc, n, m);    //释放 huffman 编码内存
    cout << endl;
    out << endl;
    //system("pause");
}

//使用仿射密码进行加密解密
int canshu(int k, int b)
{

```

```

int c = 1, t, mod = 26;    //模 26
if (k == 1 || k % 2 == 0)
    return 0;
if (mod < k)
{
    t = mod;
    mod = k;
    k = t;
}
while (c != 0)
{
    c = mod % k;
    mod = k;
    k = c;
}
if (mod == 1)
    return 1;
else
    return 0;
}

//仿射加密
string FSjiami(string str, int k, int b)
{
    int i;
    //cout << "\n 输入的需要加密的明文为:" << str << endl;
    for (i = 0; i < str.length(); i++)
    {
        int a;
        //a = (str[i] - 97) * k + b;    //加密
        a = (str[i]) * k + b;
        if (a > 25)    //对照为 0-25    mod 为 26
        {
            do    //保证不超出域
            {
                a = a - 26;
            } while (a > 25);
            a = a + 97;    //加密后得到的 ASCII 码;
            str[i] = (char)a;
        }
        else
        {
            a = a + 97;
            str[i] = (char)a;
        }
    }
}

```

```

    }
}

return str;
}

//仿射解密
string FSjiemi(string pass, int k, int b)
{
    int i;
    for (i = 0; i < pass.length(); i++)           //解密过程
    {
        int d;
        d = (pass[i] - 97) - b;
        while (d % k != 0)
        {
            d = d + 26;
        }
        d = d / k;
        pass[i] = (char)(d + 26);
    }

    return pass;
}

```

//这里是标准生成矩阵, 所以省略了前四列的单位阵, 此矩阵 4 行 3 列

```
int G[4][3];
```

//这里是译码表, 由于 (7, 4) 系统循环码最小码距 d=3, 纠错能力 t=1;

//故这里只有 8 行

```
string S[8][16];
```

//初始化标准生成矩阵

```

void create_G()
{
    //初始化生成矩阵, 同时, 生成阵的转置为监督阵
    G[0][0] = 1; G[0][1] = 0; G[0][2] = 1;      //第一行  1 0 1
    G[1][0] = 1; G[1][1] = 1; G[1][2] = 1;      //第二行  1 1 1
    G[2][0] = 1; G[2][1] = 1; G[2][2] = 0;      //第三行  1 1 0
    G[3][0] = 0; G[3][1] = 1; G[3][2] = 1;      //第四行  0 1 1
}

```

```

//初始化译码表
void create_S()
{
    //初始化译码表第一行, 第一行都为需用码字
    S[0][0] = "0000000"; S[0][1] = "1111111"; S[0][2] = "0001011"; S[0][3] = "0010110";
    S[0][4] = "0101100"; S[0][5] = "1011000"; S[0][6] = "0110001"; S[0][7] = "1100010";
    S[0][8] = "1000101"; S[0][9] = "0011101"; S[0][10] = "0111010"; S[0][11] = "1110100";
    S[0][12] = "1101001"; S[0][13] = "1010011"; S[0][14] = "0100111"; S[0][15] = "1001110";
    for (int i = 0; i < 7; i++)
    {
        for (int j = 0; j < 16; j++)    //循环生成所有可以纠错的图样
        {
            int x = 0; string s = "", befor = "", after = "";
            befor = S[0][j].substr(0, i);
            after = S[0][j].substr(i + 1, (S[0][j].length()) - i - 1);
            s = S[0][j].substr(i, 1);
            x = stoi(s);    //string 转 int
            x = (x + 1) % 2;
            s = std::to_string(x);    //int 转 string

            S[i + 1][j] = befor + s + after;
        }
    }
}

```

//二进制字符串转 10 进制, 主要是使信道译码之后的字符串变成 ASCII 码的字符串, 方便转换密文

```

int bin_to_deci(string inin)
{
    int len = inin.length();
    int n = 0;
    for (int i = 0; i < len; ++i)
    {
        //其中注意 if 的判断两个条件相等的符号
        if (inin.substr(i, 1) == "1")
            n += pow(2, len - 1 - i);
    }
    return n;
}

```

//十进制转二进制 (八位二进制)

```

string deciToBin_8bit(int deci)

```

```

{
    string hexStr(8, ' ');
    int Value = 0;
    int i = 0;
    for (; deci != 0; ++i, deci /= 2)
    {
        Value = deci % 2;
        hexStr.at(i) = Value + '0';
    }
    hexStr = hexStr.substr(0, i);
    reverse(hexStr.begin(), hexStr.end());
    if (hexStr.length() < 8)
    {
        string add(8 - hexStr.length(), '0');
        hexStr = add + hexStr;
    }
    return hexStr;
}

```

//把密文转化位 ASCII 码方便信道编码（转成 8 位 2 进制）

```

void string_to_ascii(string info, string a[])
{
    string output = "", x;
    char c; int n;
    for (int i = 0; i < info.length(); i++)
    {
        c = info[i];
        n = (int)c;
        output = deciToBin_8bit(n);
        a[i] = output;
    }
}

```

//信道编码，这里的 infomation 是 8 位二进制

```

string channel_encode(string infomation)
{
    string info, e_info = "", v1, v2, v3, v4;
    int n1 = 0, n2 = 0, n3 = 0;

    for (int i = 0; i < 2; i++)
    {
        if (i == 0) info = infomation.substr(0, 4);
        if (i == 1) info = infomation.substr(4, 4);
    }
}

```

```

        v1 = info.substr(0, 1);
        v2 = info.substr(1, 1);
        v3 = info.substr(2, 1);
        v4 = info.substr(3, 1);
        if (v1 == "1") { n1 += G[0][0]; n2 += G[0][1]; n3 += G[0][2]; }
        if (v2 == "1") { n1 += G[1][0]; n2 += G[1][1]; n3 += G[1][2]; }
        if (v3 == "1") { n1 += G[2][0]; n2 += G[2][1]; n3 += G[2][2]; }
        if (v4 == "1") { n1 += G[3][0]; n2 += G[3][1]; n3 += G[3][2]; }

        n1 = n1 % 2; n2 = n2 % 2; n3 = n3 % 2;
        v1 = std::to_string(n1); v2 = std::to_string(n2); v3 = std::to_string(n3);
        e_info = e_info + info + v1 + v2 + v3;
        n1 = 0; n2 = 0; n3 = 0;
    }
    return e_info;    //输出 14 位，也就是两个码字合并
}

//信道译码，输入 14 位，两个码字
string channel_decode(string infomation)
{
    string info, de_info = "";

    for (int x = 0; x < 2; x++)
    {
        if (x == 0) info = infomation.substr(0, 7);
        if (x == 1) info = infomation.substr(7, 14);
        for (int i = 0; i < 8; i++)
        {
            for (int j = 0; j < 16; j++)
            {
                if (S[i][j] == info)
                {
                    de_info += info.substr(0, 4);
                    goto goon;
                }
            }
        }

        goon::;
    }

    return de_info;    //8 位，包含两个四位信息源，其余六位校验位已经删去
}

```

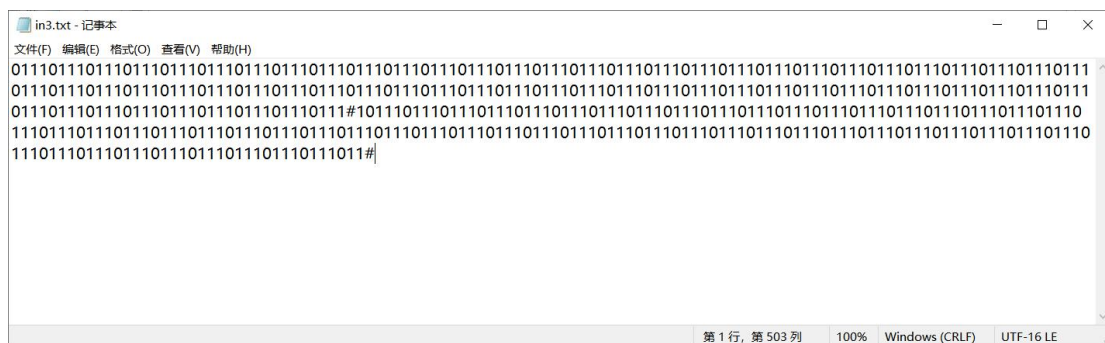
```

//模拟 BSC 信道传输
string BSC(string infomation)
{
    string output = "", X;
    for (int i = 0; i < infomation.length(); i++)    //一个字符一个字符判断
    {
        srand((int)time(0));
        X = infomation.substr(i, 1);
        if (X == "0")
        {
            if (10 == rand() % 100)    //p=10 的负 2 次
                X = "1";
            goto gon;    //防止此处 X 重新赋值为 1 后进入到下一个判断
        }
        if (X == "1")
        {
            if (100 == rand() % 100)    //p=10 的负 2 次
                X = "0";
            goto gon;
        }
        gon:output += X;
    }
    return output;
}

```

4.运行结果：输入、输出及结果分析

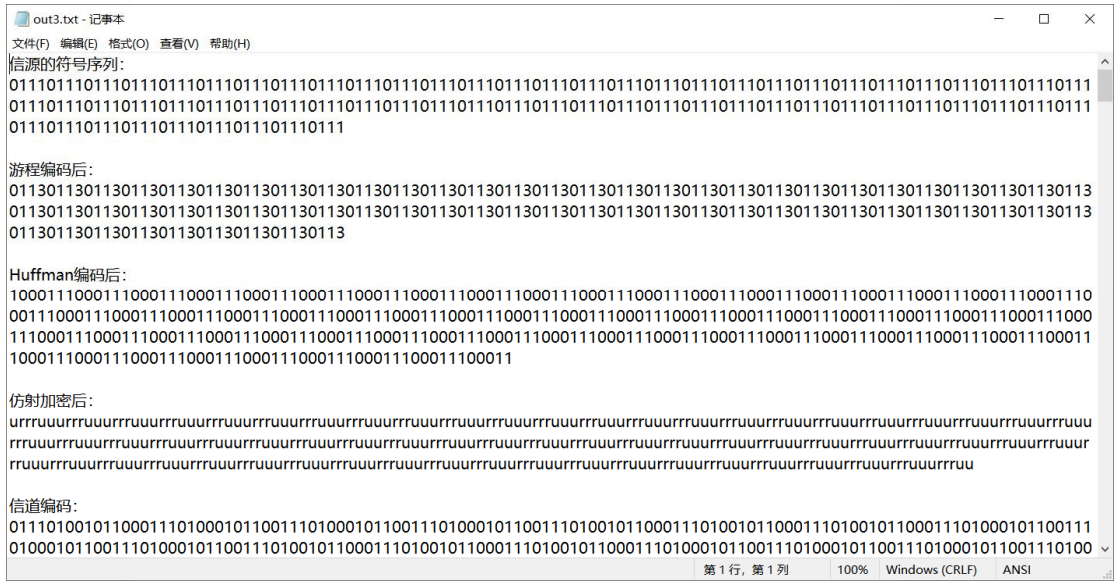
4.1 输入文件 in3.txt



含两组输入数据，均符合条件：连续出现的 0 或 1 序列的长度不超过 16，n 不小于 256。

4.2 输出文件 out3.txt

第一组输出:



[illegible]

第二组输出:

[illegible][illegible]



4.4 结果分析

本次设计，加密的参数需要用户自行输入；发送端输入的两串符号序列从文件 in3.txt 中获得；游程编码结果表示为“0”+“0 游程长度”和“1”+“1 游程长度”交替；Huffman 编码由游程编码结果动态建立 Huffman 树得到；加密采用仿射加密结果由字母表示；同时还输出了信道编码、信道传输的模拟传递串、信道译码、仿射解密、Huffman 解密、游程解密的结果。

本次设计中游程编码结果用 Huffman 编码的效率很低，可改进的地方：

- 1.对游程编码的结果可以分为“0 游程”和“1 游程”两部分，在 Huffman 编码的时候参考 MH 编码的思路，首先测定“0”游程长度和“1”游程长度的概率分布，即以游程长度为元素，构造一个新的信源，分别进行 Huffman 编码，查表编码及译码，提高编码效率。

- 2.信道传输过程噪声模拟可以简化一些。

三、 设计体会

通过本次课程设计，学会了算术编码基本流程，基本能够调试简单算术编码程序；了解了游程编码，Huffman 编码，加密编码以及信道编码的原理和基本流程；了解了编码与译码，加密与解密的基本原理，完成过程中通过查阅资料了解了对称与非对称加密算法的一些简单知识，并在课程设计过程中使用了仿射加密方法；深入了解了霍夫曼树和其叶子节点的生成，以及权值的赋值。

本次课程设计实验，提高了我查阅资料和文献检索的能力；对信息论相关知识有了进一步的理解，逐渐建立综合编码的思想；同时提高了自己解决问题的能力；对编程能力也有很大的提高；这样类似解决更加接近实际生产生活问题的过程，对我今后的学习和工作也大有裨益。本次课程设计实验，锻炼了我独立分析问题以及解决问题的能力，也让我深刻认识到了自己编程能力的不足，在以后的学习中要加强自己的编程能力的提高。